ITA0329 – MOBILE COMPUTING

ASSIGNMENT – 5

ANDROID APPLICATION FEATURES & PERFORMANCE ENGINEERING

STUDENT DETAILS	
Name	V. Naga Nischay
Register Number	192472255
Course	ITA0329 – Mobile Computing
Assignment	Assignment 5 – CO5

PART A – DESIGN, ANALYSIS & ARCHITECTURAL REASONING

TASK A1 – DESIGN A CONSTRAINT-AWARE INTENT & SERVICE ARCHITECTURE

A1.1 – Model the Problem

The application monitors a patient's health sensors, stores readings locally, uploads eligible records to a backend and informs the user when a critical event occurs. The design separates time-sensitive alert handling from deferrable bulk upload work.

Inputs

- Sensor events: timestamped heart-rate, temperature or oxygen-level readings, together with sensor identifier and quality metadata.
- Network state: connected or disconnected, metered or unmetered, and an estimate of connection quality.
- Device state: battery percentage, charging status, Doze/background restriction state and available storage.
- User and security state: permissions, authentication session, notification preference, consent and encryption-key availability.

Outputs

- Encrypted upload acknowledgements and server-side records.
- Locally retained pending records when upload is not currently feasible.
- User notifications for critical events and meaningful synchronisation status.
- Audit events describing scheduling, deferral, cancellation and execution outcomes.

Decision at each stage: accept or reject a sensor event; persist it locally; classify it as critical or routine; choose immediate execution, a foreground service or a WorkManager request; check constraints; upload, defer or cancel; and notify only through an authorised channel.

Goal condition: every valid reading is durably stored; every critical event is surfaced within the application's permitted alert policy; eligible records are eventually acknowledged by the backend; and no unauthorised component or network endpoint can read sensitive data.

A1.2 – Define the Runtime State

The runtime state must be sufficient to resume safely after process death, network loss, device restart or a worker retry. State should be persisted rather than held only in memory.

State field	Purpose
record_id, patient_id, timestamp	Uniquely identify a reading and preserve event ordering.
payload_reference, sensitivity	Point to encrypted local content and classify privacy requirements.
status	CAPTURED, QUEUED, RUNNING, DEFERRED, ACKNOWLEDGED, FAILED or CANCELLED.
priority, retry_count, next_retry_at	Support critical-first scheduling and bounded retry/backoff.
network_snapshot, battery_snapshot	Record the conditions under which a scheduling decision was made.
permission_state, session_id	Prevent work from proceeding after permission revocation or logout.
encryption_key_version, checksum	Support encryption lifecycle and integrity verification.
worker_id, lease_expiry	Prevent duplicate concurrent processing and recover abandoned work.

A worker obtains a short-lived lease on a record before uploading it. The lease and idempotency key prevent two workers from sending the same item concurrently. After a process restart, expired leases are returned to QUEUED or DEFERRED, depending on current constraints.

A1.3 – Pruning / Feasibility Conditions

- Reject an upload if the user is logged out, consent is absent, the required permission has been revoked, or the encryption key is unavailable.
- Defer routine bulk uploads when the network is unavailable, the connection is too weak, the device is in Doze, or battery is below the configured threshold and the device is not charging.
- Do not discard a valid health record merely because upload is temporarily impossible. Retain it encrypted until a safe retry or an authorised retention-policy deletion.
- Cancel an in-flight operation on logout or consent withdrawal, revoke its worker lease, clear transient plaintext buffers and prevent subsequent network transmission.
- Critical alert evaluation is not pruned by the bulk-upload rule. A locally detected critical event follows the approved alert path, subject to permissions and platform notification restrictions.

The feasibility test is therefore a safety gate, not simply a performance preference. A low battery can justify deferring routine upload, but it cannot justify deleting a valid clinical record.

A1.4 – Complete Architectural Pseudocode

The following language-independent design shows work selection, Intent choice, feasibility checks, execution, result handling and cancellation.

```
PROCEDURE PROCESS_BACKGROUND_QUEUE(state)
  WHILE state.application_active
    item ← SELECT_NEXT_ITEM(state.queue)
    IF item = NONE
      WAIT_FOR_EVENT(sensor, network, battery, user)
      CONTINUE
    END IF

    IF NOT state.user_authenticated OR NOT state.consent_valid
      CANCEL(item)
      WIPE_TRANSIENT_PLAINTEXT(item)
      RECORD_AUDIT("cancelled: authentication/consent")
      CONTINUE
    END IF

    IF item.permission_required AND NOT state.permission_granted
      MARK(item, FAILED_PERMISSION)
      NOTIFY_USER_IF_APPROPRIATE()
      CONTINUE
    END IF

    IF item.type = CRITICAL_ALERT
      intent ← EXPLICIT_INTENT(AlertReceiver)
      EXECUTE_ALERT_EVALUATION(item, intent)
      CONTINUE
    END IF

    IF item.type = ROUTINE_UPLOAD
      intent ← EXPLICIT_INTENT(UploadWorker)
      IF NOT ENCRYPTED(item) OR NOT KEY_AVAILABLE(item)
        MARK(item, BLOCKED_SECURITY)
        CONTINUE
      END IF
      IF NOT NETWORK_AVAILABLE(state) OR
        (state.battery < MIN_BATTERY AND NOT state.charging) OR
        state.doze_active
        MARK(item, DEFERRED)
        ENQUEUE_WORKMANAGER(item, required_constraints)
        CONTINUE
      END IF
      ACQUIRE_IDEMPOTENT_LEASE(item)
      result ← EXECUTE_UPLOAD_WORKER(item, intent)
      IF result = ACKNOWLEDGED
        MARK(item, ACKNOWLEDGED)
        RELEASE_LEASE(item)
      ELSE
        ROLLBACK_PARTIAL_UPLOAD_IF_SUPPORTED(item)
        INCREMENT_RETRY(item)
        SCHEDULE_EXPONENTIAL_BACKOFF(item)
```

```
        END IF
      END IF
    END WHILE
  END PROCEDURE
```

Explicit Intents are used for internal receivers and workers because the destination is known. An implicit Intent is reserved for a user-facing action such as opening a supported map or dialler

A1.5 – Trace the Design

Scenario: the patient device collects routine readings, loses connectivity, enters Doze, detects a critical event and then receives a logout command.

Event	**Constraint check and decision**	**Result**
1. Sensor reading received	Validate sensor source and timestamp; encrypt and persist locally.	Record R1 becomes QUEUED.
2. Network disconnects	Network feasibility is false for upload.	R1 is DEFERRED; no repeated network attempt.
3. Device enters Doze	Routine background execution is restricted.	WorkManager retains constrained work for a suitable window.
4. Critical reading detected	Evaluate local threshold and alert policy; do not wait for bulk sync.	Critical alert is generated through the permitted notification path.
5. User logs out	Authentication and consent/session gate becomes false.	Cancel pending user-scoped upload, release leases and wipe transient plaintext.

Trace interpretation

The design chooses work according to priority and type, checks constraints before execution, defers only work that can safely wait, and treats logout as a cancellation boundary. The critical event is handled separately from routine upload. If an upload had already reached the backend, the client must use an idempotency key and server-side status check rather than assuming that cancellation can reverse a completed remote transaction.

The architecture remains responsive because long-running network operations are not performed on the main thread. WorkManager is used for deferrable, persistent work; a foreground service is reserved for user-noticeable ongoing activity where platform policy permits it.

TASK A2 – DON'T SYNCHRONISE WHAT CANNOT BE SAFE

A2.1 – Diagnose Computational and Security Waste

The naïve SyncAll algorithm attempts every local record without first checking whether it is eligible. It can repeatedly wake the radio, establish connections for records that cannot be sent, consume battery on metered or weak networks and waste CPU on records that are not encrypted or are no longer authorised.

- Unnecessary work: records already acknowledged, expired, duplicated or unchanged may be uploaded again.
- Energy drain: one network request per record creates repeated radio wake-ups and connection overhead.
- Security exposure: sensitive plaintext may be transmitted if encryption status is not checked before upload.
- Conflict risk: the algorithm has no version, timestamp or conflict-resolution policy.
- Poor recovery: continuing after a network failure does not record retry state, backoff or partial completion.
- Privacy risk: logout or consent withdrawal is not treated as a cancellation condition.

A2.2 – Incremental, Constraint-Aware Synchronisation

```
PROCEDURE INCREMENTAL_SYNC(local_store, device_state)
  sync_set ← EMPTY
  FOR EACH record IN local_store ORDER BY priority DESCENDING
    IF record.status = ACKNOWLEDGED OR record.is_expired
      CONTINUE
    END IF
    IF NOT device_state.authenticated OR NOT record.consent_valid
      DISCARD_OR_QUARANTINE(record, policy)
      CONTINUE
    END IF
    IF NOT record.encrypted OR NOT record.integrity_valid
      QUARANTINE(record)
      CONTINUE
    END IF
    IF record.remote_version CONFLICTS WITH record.local_version
      RESOLVE_CONFLICT(record, version_policy)
      CONTINUE
    END IF
    IF NOT NETWORK_QUALITY_SUFFICIENT(record.size)
      DEFER(record, "network")
      CONTINUE
    END IF
    IF device_state.battery < threshold AND NOT device_state.charging
      DEFER(record, "battery")
      CONTINUE
    END IF
    ADD record TO sync_set
    IF sync_set.size >= batch_limit
      BREAK
```

```
    END IF
  END FOR
  UPLOAD_BATCH(sync_set, idempotency_keys)
  UPDATE_ACKNOWLEDGEMENTS_AND_RETRY_STATE()
END PROCEDURE
```

The alternatives are explicit: Upload when all checks pass; Defer when a temporary condition prevents safe execution; Discard only when a valid retention or user-rights policy requires deletion; Resolve Conflict when local and remote versions differ.

A2.3 – Demonstrate Pruning

Assume a device has 35% battery, is not charging, and has a weak metered connection. The routine-upload battery threshold is 25%; a minimum network quality score of 0.6 is required. The following records are illustrative.

Record	Priority	Size	Sensitivity	Decision
R1 – routine summary	Low	8 KB	Low	Defer: network quality is below 0.6.
R2 – critical alert	Critical	2 KB	High	Use secure urgent alert path; upload only when policy permits.
R3 – raw sensor batch	Medium	4 MB	High	Defer: weak network and large payload.
R4 – encrypted status	High	12 KB	Medium	Defer: network condition fails.
R5 – expired duplicate	Low	8 KB	Low	Discard under documented retention/deduplication rule.

Early termination example: when the queue reaches R3 and the network score is 0.2, the scheduler can defer the remaining routine batch rather than repeatedly testing and attempting uploads over the same known-bad connection. This branch cannot produce a safe successful upload under the current network feasibility rule. The records remain queued, so correctness is preserved.

A2.4 – Challenge the Pruning Rule

If offline-first operation with eventual consistency is required, the assumption that a record must have an acceptable network before it can enter the synchronisation workflow is too restrictive. The application must accept local writes while offline, assign stable identifiers and version metadata, and queue changes for later reconciliation.

The safe change is not to remove encryption or authentication checks. Instead, separate local acceptance from remote transmission. A record may be committed to the encrypted local store even when network quality is zero. Synchronisation is deferred, not rejected. Conflict resolution must use a defined policy such as version vectors, server revision numbers or domain-specific merge rules. Eventual consistency means replicas may temporarily differ, but they must converge when communication resumes and conflicts are resolved.

TASK A3 – DISCOVER THE MOBILE-BACKGROUND DESIGN PATTERN

A3 – Comparison of Three Background Problems

Sensor collection, offline synchronisation and push-notification delivery appear different, but all are event-driven background workflows. Each has a persistent state, a set of candidate actions, feasibility constraints, a completion condition and a recovery path.

Design element	**Sensor collection**	**Data synchronisation**	**Push notification**
Component	Sensor API + foreground service when user-visible; scheduled worker for deferrable sampling.	WorkManager worker and repository layer.	Push messaging receiver/service and notification manager.
State	Last sample time, sensor permission, sampling interval, buffered readings.	Pending records, versions, retry count, auth and key state.	Message ID, delivery state, expiry, user preference and channel.
Decision variable	Sample now, wait, reduce sampling or stop.	Upload, defer, merge conflict or discard by policy.	Display, suppress, defer or route to in-app inbox.
Candidate choices	Start/continue/stop sampling; adjust interval.	Batch upload; retry; resolve; defer.	Show notification; group; mark read; ignore expired.
Feasibility	Permission, battery, sensor availability, foreground-service policy.	Network, battery, authentication, encryption, integrity.	Permission, channel, user preference, message validity.
Goal	Valid reading stored with timestamp and provenance.	Replicas converge and acknowledged records are tracked.	Valid message reaches the user through an allowed channel.
Pruning/deferral	Skip redundant sample or lower frequency under low power.	Defer routine batch when network/battery is unsuitable.	Suppress expired, duplicate or disallowed message.
Undo/cancel	Stop sensor listener and release resources.	Cancel worker, release lease and retain safe local copy.	Cancel pending notification or update its state.

A3.1 – Common Computational Pattern

All three workflows can be modelled as a state-transition system. An event arrives, the application updates state, selects a candidate action, tests constraints and either executes, defers or cancels. The workflow must be idempotent because mobile processes can be killed and events can be delivered more than once. It must also be observable through logs and counters so that failures can be diagnosed.

A3.2 – Generic Background-Work Framework

```
PROCEDURE SCHEDULE_OR_EXECUTE(state)
  event ← GET_NEXT_PENDING_EVENT(state)
  IF event = NONE
    RETURN IDLE
  END IF

  UPDATE_PERSISTENT_STATE(state, event)
  action_set ← GENERATE_CANDIDATE_ACTIONS(event, state)

  FOR EACH action IN ORDER_BY_PRIORITY(action_set)
    IF NOT PRECONDITIONS_VALID(action, state)
      RECORD_DECISION(action, "infeasible")
      CONTINUE
    END IF

    IF TEMPORARY_CONSTRAINT_BLOCKS(action, state)
      PERSIST_DEFERRED_ACTION(action, retry_condition)
      CONTINUE
    END IF

    token ← ACQUIRE_IDEMPOTENCY_TOKEN(action)
    result ← EXECUTE_OFF_MAIN_THREAD(action, token)

    IF result = SUCCESS
      COMMIT_STATE_TRANSITION(action)
      RELEASE_RESOURCES(action)
      RETURN COMPLETED
    ELSE
      RESTORE_SAFE_LOCAL_STATE(action)
      RELEASE_RESOURCES(action)
      APPLY_RETRY_OR_CANCEL_POLICY(action, result)
    END IF
  END FOR
  RETURN DEFERRED_OR_FAILED
END PROCEDURE
```

A3.3 – Role of Undo and Cancellation

Undo and cancellation restore invariants after partial execution. Consider a synchronisation worker that uploads a large batch and then loses connectivity. Without rollback or reconciliation, the local application might mark all records as sent even though only part of the batch reached the server. The result would be missing data or duplicates on retry.

A robust design records per-record acknowledgements, uses idempotency keys, releases locks and leases, removes temporary files, and leaves unacknowledged records in a retryable state. A remote operation that has already committed cannot always be literally undone; the correct recovery is a compensating action or a status reconciliation with the backend.

The common pattern is therefore: persist intent, validate, execute, commit only confirmed outcomes, and recover all uncommitted state.

TASK A4 – TRACTABILITY: LOCAL VS CLOUD VS HYBRID PROCESSING

A performance problem must be described using workload, latency target, resource budget, correctness requirement and operating conditions. “Runs slowly” does not identify which stage is slow, how often it occurs, what target is missed or whether the bottleneck is CPU, memory, network, battery or synchronisation.

Problem	**What constitutes a solution?**	**Efficiently verified?**	**Resource class and reason**
On-device tiny image classification	Model returns the correct class within an acceptable latency and memory budget.	Yes: labelled test set, latency and memory profiling.	On-device feasible; small model avoids network dependency.
Cloud speech recognition	Audio is securely transmitted and returned as accurate text within an acceptable end-to-end delay.	Partly: benchmark accuracy and network latency; provider internals may be opaque.	Network-bound; upload and round-trip dominate.
Nearby peer-to-peer file sharing	Authenticated peers transfer an intact file with integrity verification.	Yes: checksum, throughput, failure and pairing tests.	Hybrid required; local radio plus discovery and security overhead.
End-to-end encrypted messaging	Only intended endpoints can decrypt messages; delivery and ordering are reliable.	Partly: protocol tests and cryptographic review; full proof is difficult.	Security-critical; key management and metadata protection matter.
Real-time multiplayer state sync	Players receive consistent-enough state with bounded latency and acceptable correction.	Yes for defined simulation; test jitter, loss and convergence.	Network-bound / hybrid required; latency and server authority interact.
Offline vector-map rendering	Visible map tiles render smoothly from cached data with bounded memory.	Yes: deterministic tile sets and frame/memory profiling.	Battery-critical / on-device feasible; large local datasets and rendering cost.

Verification and Formulation

For each problem, verification should measure both functional correctness and operational constraints. Classification accuracy alone is insufficient for image inference if memory use exceeds device limits. A successful speech-recognition response is insufficient if the network delay violates the interaction target. A file transfer is not correct merely because bytes arrived; integrity and peer authentication must also pass.

A4 – Local, Cloud and Hybrid Processing: Engineering Analysis

Why Local and Cloud Placement Matters

Local processing reduces network dependence, can improve privacy by keeping raw data on the device and offers predictable operation when offline. However, it consumes device CPU, memory and battery, and may be limited by thermal throttling or hardware diversity.

Cloud offloading moves computation to scalable infrastructure and can use larger models or centralised resources. It introduces network latency, bandwidth cost, service availability dependency and a larger data-governance surface. Sensitive information must be minimised, encrypted in transit and protected at rest with appropriate access controls.

Hybrid processing divides work according to urgency, sensitivity and resource requirements. For example, a phone can perform lightweight speech activity detection locally and send only necessary audio segments for cloud recognition. A peer-to-peer transfer can use local Wi-Fi while relying on a backend only for identity or rendezvous.

Scalability

A local design scales across users by distributing computation to devices, but performance differs by device profile and each device bears its own energy cost. A cloud design centralises resource management and can scale horizontally, but concurrent users create server, network and cost pressure. A hybrid design can reduce central workload while preserving cloud support for expensive or shared operations.

Security

Local execution is not automatically secure: a compromised or rooted device may expose stored data. Cloud execution is not automatically insecure: strong encryption, access control, retention limits and audited processing can reduce risk. The relevant question is where data is created, stored, decrypted and transmitted, and which trust boundaries are crossed.

Therefore, a meaningful performance specification includes target latency, percentile behaviour, throughput, energy per task, memory footprint, data sensitivity, network assumptions and failure behaviour.

TASK A5 – DESIGN WHEN EXACT OPTIMISATION MAY BE TOO EXPENSIVE

A5.1 – Exact Perspective

An exact search can enumerate combinations of image/video decoding resolution, cache capacity and prefetch distance. For each configuration, the renderer estimates frame time, dropped frames, memory pressure and energy cost. It then selects the feasible configuration with the lowest total cost.

```
BEST ← NONE
BEST_COST ← INFINITY
FOR EACH resolution IN RESOLUTION_LEVELS
  FOR EACH cache_size IN CACHE_OPTIONS
    FOR EACH prefetch_distance IN PREFETCH_OPTIONS
      result ← SIMULATE_FEED(resolution, cache_size, prefetch_distance)
      IF result.frame_time <= 16 ms AND
        result.memory <= MEMORY_BUDGET
        cost ← WEIGHT_FRAME_DROPS(result) +
              WEIGHT_ENERGY(result) +
              WEIGHT_MEMORY(result)
        IF cost < BEST_COST
          BEST ← configuration
          BEST_COST ← cost
    END FOR
  END FOR
END FOR
RETURN BEST
```

If there are r resolution levels, c cache sizes and p prefetch distances, one static search evaluates r × c × p configurations. If the system also considers n feed items, d device profiles and changing network/battery states, the search space grows rapidly. Re-running an exhaustive search during scrolling is too expensive and may itself cause frame drops.

A5.2 – Greedy / Adaptive Strategy

```
INITIALISE quality, cache_limit, prefetch_window
LOOP
  metrics ← MEASURE(frame_time, dropped_frames, battery, memory)
  IF frame_time > 16 ms OR memory > MEMORY_LIMIT
    LOWER quality one step
    REDUCE prefetch_window
  ELSE IF frame_time < 12 ms AND battery is healthy
    IF cache has room
      INCREASE cache_limit one step
    ELSE
      INCREASE prefetch_window one step
  ELSE
    KEEP current parameters
  END IF
  APPLY_PARAMETERS_AT_SAFE_FRAME_BOUNDARY()
  IF metrics stable for K intervals
    ENTER steady state and continue monitoring
  END IF
```

A5.2 – Selection Criterion and State Update

The adaptive controller uses a safety-first criterion. It first attempts to satisfy the frame-time and memory limits. Among feasible settings, it prefers the highest visual quality that does not create sustained frame drops or excessive battery use. It changes one or a small number of parameters per control interval to avoid oscillation.

- Current metrics: moving-average frame time, 95th-percentile frame time, dropped-frame count, memory use, battery percentage and thermal status.
- Selected parameters: decode resolution, cache limit, prefetch window and video playback quality.
- Selection criterion: restore frame-time and memory feasibility first; improve quality only when there is measured headroom.
- State update: apply changes at a frame boundary, record the previous setting and retain a short history for rollback.
- Termination/steady state: stop tuning when metrics remain within a tolerance band for K consecutive intervals; continue monitoring because workload changes.

A5.3 – What Has Been Sacrificed?

A feasible smooth-scrolling configuration satisfies operational constraints, such as keeping frame time near or below 16 ms for the tested workload. It does not prove that total energy is globally minimised for every possible quality-of-service level.

The heuristic sacrifices a global optimality guarantee. A greedy decision may reduce resolution early even though a different cache/prefetch combination would have achieved better total energy and quality. It may also settle in a local configuration because it evaluates only nearby changes.

This trade-off is acceptable in production when a fast, stable decision is more valuable than spending substantial CPU time searching for a theoretical optimum. The system should expose quality controls, retain safe defaults and use telemetry to refine the heuristic.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

TASK B1 – IMPLEMENT AND INSTRUMENT THE BACKGROUND ARCHITECTURE

The following implementation model represents a healthcare sensor-upload pipeline. It uses a worker queue, explicit feasibility checks, a FIFO baseline and a priority-based alternative. In a real Android application, WorkManager would persist eligible work and Android APIs would provide battery, connectivity and permission state.

Instrumentation Design

- enqueued_count: total work items accepted by the scheduler.
- deferred_count and cancelled_count: work postponed by temporary constraints or stopped by security/session conditions.
- success_count: uploads confirmed by an acknowledgement.
- decision_log: timestamp, item ID, network quality, battery, Doze state, permission and decision.
- peak_workers: maximum simultaneous worker count.
- backlog_count: number of valid records not yet acknowledged at the end of the run.

Illustrative Simulation Assumptions

For reproducible classroom testing, the simulation uses a 25% battery threshold for routine uploads, network quality of at least 0.6, and a maximum of two concurrent workers. A critical alert may be prioritised, but it still must pass authentication, consent and encryption checks. These are example thresholds and must be tuned for a real product.

B1.1 – Validation Scenarios

Test	Input conditions	Expected behaviour	What it verifies
Good conditions	Battery 80%, charging, network 0.95, permissions granted.	Eligible records execute and receive acknowledgements.	Successful scheduling and upload path.
Doze / low battery	Battery 15%, not charging, Doze active.	Routine work is deferred; queue remains persistent.	Constraint handling and absence of wasteful retries.
Permission revoked	Permission changes to denied while a task is pending or running.	Task stops before protected access; state is marked permission-blocked.	Boundary safety and permission revalidation.

B1.2 – Design Compared with Implementation

The conceptual design describes a single selection-and-execution loop. The implementation model adds a worker lease, idempotency key, bounded retry counter and per-record acknowledgement. These additions are necessary because mobile work may be interrupted and retried, and a request may reach the backend even if the client does not receive the response.

B1.3 – Two Scheduling Versions

Version 1 uses first-in, first-out order. It is simple and preserves arrival order, but an old low-priority batch can delay a critical alert or high-value record.

Version 2 uses a priority score: criticality contributes 5 points, urgency contributes 3 points, and age contributes 1 point per configured age interval. The scheduler sorts by score but still applies all security and resource constraints. A priority score never overrides encryption, consent or permission checks.

```
FUNCTION PRIORITY_SCORE(item, current_time)
  score ← 5 * item.criticality +
        3 * item.urgency +
        MIN(item.age_minutes / AGE_INTERVAL, AGE_CAP)
  RETURN score
END FUNCTION

SELECT_NEXT(queue, mode)
  IF mode = FIFO
    RETURN OLDEST_ELIGIBLE_ITEM(queue)
  ELSE
    RETURN ELIGIBLE_ITEM_WITH_MAXIMUM_PRIORITY_SCORE(queue)
  END IF
END
```

B1.4 – Illustrative Measurement

The table below is a sample simulation for the same 10-record workload. Both strategies see the same battery, network and permission thresholds. It is not a physical-device benchmark.

Measure	**FIFO**	**Priority heuristic**
Work items enqueued	10	10
Deferred / cancelled	4 deferred, 1 cancelled	4 deferred, 1 cancelled
Successful executions	5	5
Peak concurrent workers	2	2
Critical item completion order	6th eligible item	1st eligible item

Interpretation: the heuristic does not necessarily increase the total number of successful uploads when constraints are identical. Its benefit is the order in which valuable work is delivered. If a critical record is feasible, it is selected earlier. The total number of deferrals may remain unchanged because a priority score cannot make a weak network or revoked permission safe.

TASK B2 – ENGINEER AN ADVERSARIAL WORKLOAD

Two workloads are constructed with the same number of records and the same constraint thresholds. The difference is their structure: one contains small, eligible records, while the other contains large, sensitive records and several items that are temporarily infeasible.

Workload A – Low Deferral Instance

Record group	Count	Size	Priority / sensitivity	Conditions
Routine summaries	4	10–20 KB	Low / low	Encrypted; network suitable.
Status updates	3	5–15 KB	Medium / medium	Authenticated; network suitable.
Critical alerts	2	2 KB	Critical / high	Encrypted and consent valid.
Metadata	1	1 KB	Low / low	Valid and not expired.

Prediction: most records should pass the feasibility checks. Small payloads reduce transmission time and energy, and the network quality is assumed to remain above the threshold. Only records affected by a deliberate test event should be deferred.

Workload B – High Deferral Instance

Record group	Count	Size	Priority / sensitivity	Conditions
Raw sensor batches	4	3–5 MB	High / high	Weak network; encryption key rotation pending.
Routine summaries	3	20 KB	Low / low	Battery 18%, not charging.
Critical alerts	2	2 KB	Critical / high	One has consent withdrawn.
Metadata	1	1 KB	Medium / medium	One duplicate/expired record.

Prediction: Workload B should require more deferrals and security rejections. The structural causes are large payloads, weak connectivity, low battery, key availability and consent changes. The number of records is unchanged, but the amount of work per record and the proportion of infeasible candidates are much higher.

Execution and Interpretation

A representative simulation outcome is: Workload A processes 10 records, acknowledges 9 and defers 1 during a brief network interruption. Workload B processes 10 records, acknowledges 3, defers 5 and cancels or quarantines 2 for security/policy reasons. These are illustrative expected outputs, not measured results from a device.

Input size alone is insufficient to describe computational effort. Record size, sensitivity, retry behaviour, network quality, battery state, conflict rate and cancellation timing determine how much CPU, radio time and energy are consumedTASK B3 – EXACT VS APPROXIMATE UI / GRAPHICS OPTIMISATION

A simplified feed renderer is modelled using three parameters: decode quality (low, medium or high), cache size (small, medium or large) and prefetch distance (1, 2 or 3 items). The exact method enumerates every combination for small test cases. The adaptive method makes a local change based on frame-time and battery feedback.

Cost Function

For this investigation, the illustrative cost is defined as:

Cost = 5 × dropped frames + 2 × energy units + 0.5 × memory units + quality penalty.

A configuration is feasible only if estimated frame time is at most 16 ms and memory use is within the device budget. The weights express the example application's preference for smoothness while still considering energy and visual quality.

Solution A – Exact Enumeration

```
best_configuration ← NONE
best_cost ← INFINITY
FOR EACH quality IN {LOW, MEDIUM, HIGH}
  FOR EACH cache IN {SMALL, MEDIUM, LARGE}
    FOR EACH prefetch IN {1, 2, 3}
      result ← EVALUATE(quality, cache, prefetch, workload)
      IF result.frame_time <= 16 AND result.memory <= budget
        cost ← COMPUTE_COST(result)
        IF cost < best_cost
          best_cost ← cost
          best_configuration ← (quality, cache, prefetch)
        END IF
      END IF
    END FOR
  END FOR
END FOR
RETURN best_configuration, best_cost
```

Solution B – Adaptive Approximation

```
configuration ← SAFE_DEFAULT
REPEAT
    metrics ← MEASURE_FEED()
    IF metrics.frame_time > 16 OR metrics.memory > budget
        LOWER_DECODE_QUALITY()
        REDUCE_PREFETCH()
        EVICT_LRU_CACHE()
    ELSE IF metrics.frame_time < 12 AND metrics.battery > 40
        RAISE_QUALITY_IF_MEMORY_ALLOWS()
    END IF
    APPLY_AT_NEXT_FRAME_BOUNDARY()
UNTIL stable_for_required_intervals
```

RETURN configuration

B3.1 – Correctness on Small Instances

The following three small test instances illustrate how the adaptive configuration can be compared against exhaustive enumeration. Values are illustrative cost-model outputs.

Instance	Exact best cost	Adaptive cost	Match?	Explanation
1 – light feed	18	18	Yes	Low load; safe default is already optimal.
2 – mixed images	24	26	No	Adaptive method retains a larger cache than the exact optimum.
3 – video-heavy	31	31	Yes	Frame-time pressure triggers quality reduction and prefetch control.

B3.2 – Counter-example

Consider a video-heavy feed where high decoding quality causes a short initial frame-time spike, but a large cache would avoid repeated decoding later. The adaptive controller observes the spike and immediately lowers quality and reduces prefetch. It never tests the combination of medium quality, large cache and moderate prefetch, which has a lower total cost over the full session.

The decision that leads away from the optimum is the local response to the first frame-time measurement. It optimises the immediate observation rather than the entire future workload. This demonstrates that a greedy controller can be feasible and responsive without being globally optimal.

B3.3 – Increase Problem Size

With three quality levels, three cache sizes and three prefetch choices, exhaustive search evaluates 27 configurations per profile. With 6 quality levels, 8 cache sizes and 5 prefetch choices, it evaluates 240 configurations. If 20 device profiles and 10 workload states are considered, the evaluation count becomes 48,000.

The adaptive controller may perform a small number of measurements and parameter changes per control interval. It therefore reduces configuration evaluations, but its operation count depends on how frequently it adjusts and how long it takes to stabilise.

B3.4 – Engineering Trade-off

Approximation gains lower decision overhead and can react to live device conditions. It sacrifices the guarantee of finding the minimum-cost configuration. It is appropriate for continuous scrolling, where a safe and stable setting must be selected quickly and the workload changes while the user interacts with the application.

TASK B4 – FINAL ENGINEERING DECISION: MULTIMEDIA FRAMEWORK SELECTION

B4.1 – Practical Encoding Strategy

A production application must choose resolution, bitrate and upload timing for devices with different camera, CPU, battery and network capabilities. A practical heuristic uses device capability and current conditions to choose from a small set of supported profiles.

Device profile	Network	Battery	Chosen resolution	Bitrate	Upload decision
Low-end phone	Weak / metered	22%, not charging	480p	0.8 Mbps	Defer bulk upload; retain encrypted clip.
Mid-range phone	Good Wi-Fi	65%	720p	2.0 Mbps	Upload after encoding if thermal state is normal.
Flagship phone	Strong Wi-Fi	88%, charging	1080p	4.5 Mbps	Upload promptly using background transfer.

The strategy estimates encoding time from a device-class profile and estimates upload time as clip size divided by effective throughput. Energy impact is represented as a relative estimate rather than a measured joule value. A real implementation should obtain codec support and performance from platform APIs and measure on representative devices.

B4.2 – Challenge the Strategy

A possible counter-example is a flagship device with high battery but severe thermal throttling and a weak uplink. A capability-only rule may select 1080p at a high bitrate. Encoding takes longer, the device heats up and upload time becomes excessive. A lower resolution and bitrate could deliver the clip sooner with lower energy.

The heuristic should therefore include thermal status, recent encode throughput, estimated upload duration and battery trend. It should also permit a user-selected quality ceiling and avoid changing quality mid-clip unless the recording format supports it safely.

B4.3 – Production Decision Note

- Why not exhaustive search: encoding parameters, codecs, frame rates, device models and network conditions create a large search space. Exhaustive evaluation for every upload would add latency and energy overhead.
- Guarantee: the heuristic guarantees only that selected settings pass defined safety and capability checks. It does not guarantee global minimum energy or best visual quality.
- Advantage: a small candidate set and rule-based selection reduce computation and make decisions quickly using live device state.

- When exact/cloud-assisted methods fit: offline batch processing, high-value media, server-side transcoding or a stable network may justify a more expensive search or cloud optimisation.
- Evidence to monitor: encode time, dropped frames, output quality, bitrate, upload completion time, retry rate, battery drain, thermal throttling, memory use and failure rate by device class.

CONCLUSION

The assignment demonstrates that mobile engineering is a constraint-aware decision problem. Background work must preserve security and correctness while adapting to battery, network and platform restrictions. Synchronisation must distinguish temporary deferral from permanent rejection and must support conflict resolution. UI and multimedia optimisation must balance quality, responsiveness, energy and memory.

Exact optimisation is useful for small, stable problem instances and for validating heuristics. Adaptive methods are more practical when decisions must be made continuously under changing device conditions, but they do not provide a global optimality guarantee. Production decisions should be based on instrumentation across representative devices and workloads.

IMPLEMENTATION EVIDENCE TO ATTACH

- Android Studio project source code and Gradle configuration.
- Logcat output showing enqueue, defer, cancel, success and retry decisions.
- Tests for good connectivity, Doze/low battery and permission revocation.
- Profiler or benchmark evidence for frame time, memory and CPU usage.
- Device-specific multimedia encode and upload measurements.

Note: The assignment brief requires implementation evidence. The simulation tables in this document are clearly identified as illustrative; replace or supplement them with actual test outputs before final submission.